

MDTS-4214 Assignment-6

Subhayan Biswas

March 19, 2026

1 Problem Set 6

To demonstrate the application of subset selection methods, Consider the “Hitters” data. For loading the data you need to install ISLP library. We wish to predict a baseball player’s salary on the basis of various predictors. So first we need to choose an optimal model.

```
[29]: import pandas as pd
import numpy as np
import statsmodels.api as sm
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
from ISLP import load_data
from ISLP.models import ModelSpec as MS
import itertools
```

1. Clean the data by removing all rows that have missing value in any variable. You will be working with this clean data.

```
[30]: hitters = load_data('Hitters').dropna()
```

```
[31]: design = MS(hitters.columns.drop('Salary'))
X = design.fit_transform(hitters)
y = hitters['Salary']
```

2. Split the data randomly into train-test keeping 80% as training sample and remaining as test sample.

```
[32]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
↳ random_state=42)
```

```
[33]: def fit_model(features):
    model = sm.OLS(y_train, X_train[list(features)]).fit()
    return model
```

3. Perform the best subset selection on the training sample. While choosing the best model fit up to a 19 variable model. Choose the best model according to adjusted R squared and Cp values. In case the best model differs according to the selection criteria, report the best fitted model according to each selection criterion.

```
[34]: max_vars = 8
best_models = []
predictors = X.columns.drop('intercept')
for k in range(1, max_vars + 1):
    subsets = list(itertools.combinations(predictors, k))
    best_r2_model = None
    best_r2 = -np.inf

    for subset in subsets:
        feature_set = ['intercept'] + list(subset)
        model = fit_model(feature_set)
        if model.rsquared_adj > best_r2:
            best_r2 = model.rsquared_adj
            best_r2_model = {"model": model, "features": feature_set,
                            "AIC": model.aic, "BIC": model.bic, "Adj_R2":
↪model.rsquared_adj}
    best_models.append(best_r2_model)
```

4. Find the training MSE and test MSE for each of the best models selected according to different criterion as mentioned in 3 and compare their values.

```
[35]: best_bic_model = min(best_models, key=lambda x: x['BIC'])
print(f"Best Subset (BIC) Features: {best_bic_model['features']}")
```

Best Subset (BIC) Features: ['intercept', 'Hits', 'CRBI', 'Division[W]', 'PutOuts']

```
[36]: preds = best_bic_model['model'].predict(X_test[best_bic_model['features']])
print(f"Test MSE (Best Subset - BIC): {mean_squared_error(y_test, preds):.2f}")
```

Test MSE (Best Subset - BIC): 151341.81

5. Repeat 3-4 using forward stepwise selection method.

```
[37]: def forward_selection():
    current_features = ['intercept']
    remaining_features = list(predictors)

    while remaining_features:
        pvals = pd.Series(index=remaining_features, dtype=float)
        for col in remaining_features:
            model = sm.OLS(y_train, X_train[current_features + [col]]).fit()
            pvals[col] = model.pvalues[col]

        min_pval = pvals.min()
        if min_pval < 0.05:
            best_feature = pvals.idxmin()
            current_features.append(best_feature)
            remaining_features.remove(best_feature)
```

```

    else:
        break
    return current_features

```

```

[38]: fwd_features = forward_selection()
      fwd_model = fit_model(fwd_features)
      print(fwd_features, mean_squared_error(y_test, fwd_model.
      ↪predict(X_test[fwd_features])))

```

```

['intercept', 'CRBI', 'Hits', 'PutOuts', 'Division[W]', 'AtBat', 'Walks']
136744.96556330327

```

6. Repeat 3-4 using backward stepwise selection method.

```

[39]: def backward_selection():
      features = list(X.columns)
      while len(features) > 1:
          p_values = sm.OLS(y_train, X_train[features]).fit().pvalues
          p_values = p_values.drop('intercept')

          if p_values.max() > 0.05:
              features.remove(p_values.idxmax())
          else:
              break
      return features

```

```

[40]: bwd_features = backward_selection()
      bwd_model = fit_model(bwd_features)
      print(bwd_features, mean_squared_error(y_test, bwd_model.
      ↪predict(X_test[bwd_features])))

```

```

['intercept', 'AtBat', 'Hits', 'Walks', 'CRuns', 'CRBI', 'CWalks',
'Division[W]', 'PutOuts'] 128590.28411945429

```

7. Compare the results of the three methods in terms of choice of best model, train MSE and test MSE.

Comparison of Train MSE

Forward Selection: 88287.08 (lowest)

Backward Selection: 88865.02

Best Subset ($\text{Adj}R^2$): 88935.80

Best Subset (C_p): 89790.63 (highest)

Therefore, Forward selection gives the lowest training MSE, meaning it fits the training data slightly better than the others.

Comparison of Test MSE

Best Subset (C_p): 125652.3 (lowest)

Best Subset (AdjR^2): 130066.9

Forward Selection: 134001.7

Backward Selection: 134736.7 (highest)

Therefore, The Best Subset model (Cp criterion) gives the lowest test MSE, so it performs best on unseen data.

Interpretation:

Although forward selection has the lowest training MSE, its test MSE is higher indicates slight overfitting.

Backward selection performs similarly to forward but slightly worse.

Best subset selection, especially using Cp, provides the best balance between bias and variance.

Final Conclusion

The best model overall is the Best Subset Selection model chosen using Cp, because it has the lowest test MSE.

This shows that:

Evaluating all possible models (best subset) can lead to better generalization.

Stepwise methods (forward/backward) are faster but may not always give the optimal model.

2 Problem Set 7

We would be working with the Hitters dataset available in the ISLP library. We have to predict the Salary based on all available variables. First, we make sure that we delete all rows with one or more missing values.

```
[41]: from sklearn.linear_model import LinearRegression, Ridge
      from sklearn.preprocessing import StandardScaler
      from ISLP import load_data
      from ISLP.models import ModelSpec as MS
```

```
[42]: hitters2 = load_data('Hitters').dropna()
```

1. We split the data set into a training set and a test set randomly, keeping 100 observations in the test set and the remaining in the training set.

```
[43]: design2 = MS(hitters2.columns.drop('Salary'))
      X2 = design2.fit_transform(hitters2).drop('intercept', axis=1)
      y2 = hitters2['Salary']
```

```
[44]: X_train2, X_test2, y_train2, y_test2 = train_test_split(X2, y2, test_size=100,
      ↪ random_state=42)
```

```
[45]: scaler = StandardScaler()
      X_train_scaled = scaler.fit_transform(X_train2)
```

```
X_test_scaled = scaler.transform(X_test2)
```

2. We train the model using least squares and report the train MSE and test MSE.

```
[46]: ols = LinearRegression().fit(X_train_scaled, y_train2)
print(mean_squared_error(y_train2, ols.predict(X_train_scaled)),
      ↪mean_squared_error(y_test2, ols.predict(X_test_scaled)) )
```

```
69346.45379673017 144597.0560957534
```

3. We fit ridge regression models on the training set, with grid values of Lambda. We report the following:

(i) the dimension of the coefficient matrix so obtained.

(ii) The l2 norm of the regression coefficients corresponding to the 50th and 60th choice of Lambda. We comment on the output.

(iii) Test MSE corresponding to these two choices of Lambda.

```
[47]: alphas = np.logspace(10, -2, 100)
      coefs = []

      for a in alphas:
          ridge = Ridge(alpha=a).fit(X_train_scaled, y_train2)
          coefs.append(ridge.coef_)

      coef_matrix = np.array(coefs)
```

```
[48]: print(coef_matrix.shape)
```

```
(100, 19)
```

```
[49]: l2_norm_50 = np.linalg.norm(coef_matrix[49])
      l2_norm_60 = np.linalg.norm(coef_matrix[59])
```

```
[50]: print(f"\nLambda (50th): {alphas[49]:.2f} -> L2 Norm: {l2_norm_50:.2f}")
      print(f"Lambda (60th): {alphas[59]:.2f} -> L2 Norm: {l2_norm_60:.2f}")
```

```
Lambda (50th): 11497.57 -> L2 Norm: 10.29
```

```
Lambda (60th): 705.48 -> L2 Norm: 73.52
```

```
[51]: ridge_50 = Ridge(alpha=alphas[49]).fit(X_train_scaled, y_train2)
      ridge_60 = Ridge(alpha=alphas[59]).fit(X_train_scaled, y_train2)
```

```
[52]: print(mean_squared_error(y_test2, ridge_50.predict(X_test_scaled)))
      print(mean_squared_error(y_test2, ridge_60.predict(X_test_scaled)))
```

213528.76295959775
157036.9365187022

Least Squares:

Fits the data without shrinkage.

May lead to overfitting.

Reported both train MSE and test MSE.

Ridge Regression:

Shrinks coefficients toward zero using Lambda.

Higher Lambda implies smaller coefficients which again, implies higher bias, lower variance.

Results Interpretation:

The l2 norm decreases as Lambda increases:

The 50th Lambda has larger coefficients than the 60th Lambda.

Test MSE helps decide which Lambda gives better generalization.

Conclusion:

1. Ridge regression improves model stability by reducing variance.
2. Optimal λ is the one that minimizes test MSE.
3. Compared to least squares, ridge often performs better on test data.